

HOLO-RTLS

Installer & Configuration Manual

WiFi / BLE Scanner Nodes • Access Points • Tags • MikroTik

- How WiFi nodes and scanner anchors are added in software
- Server IP, port, API keys, and environment variables
- Default demo units seeded on first boot
- Bluetooth + WiFi data flow (HTTP, SSE, optional MQTT)
- Recommended hardware BOM and MikroTik networking guidance

Version 1.0 | Generated 2026-07-19 UTC | For installers & integrators

Document version: 1.0 | Generated 2026-07-19 UTC

Audience: installers, integrators, site IT, operations managers

HOLO-RTLS supports TWO positioning tracks that share the Live Map UI but use different anchor databases. Plan your site around ONE primary track or integrate both.

Track A — Location Core (UWB / hardware bridges)

Anchors: WifiNode records via /nodes or /api/nodes

Tags: hardware_id strings (e.g. TAG_001) via /trackers

Ingestion: HardwareBridge → IngestionLoop → SSE + optional MQTT publish

Best for: 10–30 cm UWB accuracy with certified gateways

Track B — WiFi/BLE RSSI Scanner

Anchors: WifiAnchor records via /api/scanner/anchors (NOT the same as WifiNode)

Tags: device WiFi MAC or BLE MAC detected by scanner Pis

Ingestion: POST /api/scanner/detections → trilateration → Tracker sync

Best for: personnel phones, BLE beacons, WiFi probes; 3–15 m accuracy

HOLO-RTLS — System Architecture

OPERATORS

Browser → Live Map / Setup / Nodes / Hardware / Trackers

HOLO-RTLS SERVER

Flask :8080 | REST API | SSE stream | SQLite/Postgres

MQTT BROKER (optional)

Mosquitto :1883 | rtls/state_changes | device RSS topics

TRACK A — Location Core

UWB bridges | WifiNode anchors | Ingestion loop

TRACK B — WiFi/BLE Scanner

Scanner Pis → POST /api/scanner/detections

UWB Tags / Gateways

Serial or MQTT position frames

BLE Beacons / Tags

iBeacon, Ruuvi, ESP32 gateway

WiFi Clients / Probes

Phones, tags, ESP32 WiFi scanner

APs*

See MikroTik note

* WiFi APs are infrastructure only — HOLO does not configure APs automatically.

Server & Network Configuration

Server network settings

Default HTTP port: 8080 (python run.py). NOT 5000.

Health check: GET `http://<server>:8080/health`

Web UI login: `http://<server>:8080/login`

API docs: `http://<server>:8080/api/docs`

Required .env variables:

`SECRET_KEY=<32+ random chars>`

`JWT_SECRET_KEY=<32+ random chars>`

Optional production:

`DATABASE_URL=postgresql://user:pass@host:5432/holo_rtls`

`MQTT_BROKER_HOST=<broker IP>`

`MQTT_BROKER_PORT=1883` (default)

`SCANNER_API_KEY=<shared secret for scanner Pis>`

Scanner Pis MUST set:

`BACKEND_URL=http://<server IP>:8080`

`SCANNER_API_KEY=<same as server>`

`ANCHOR_MAC=AA:BB:CC:DD:EE:01` (unique per Pi)

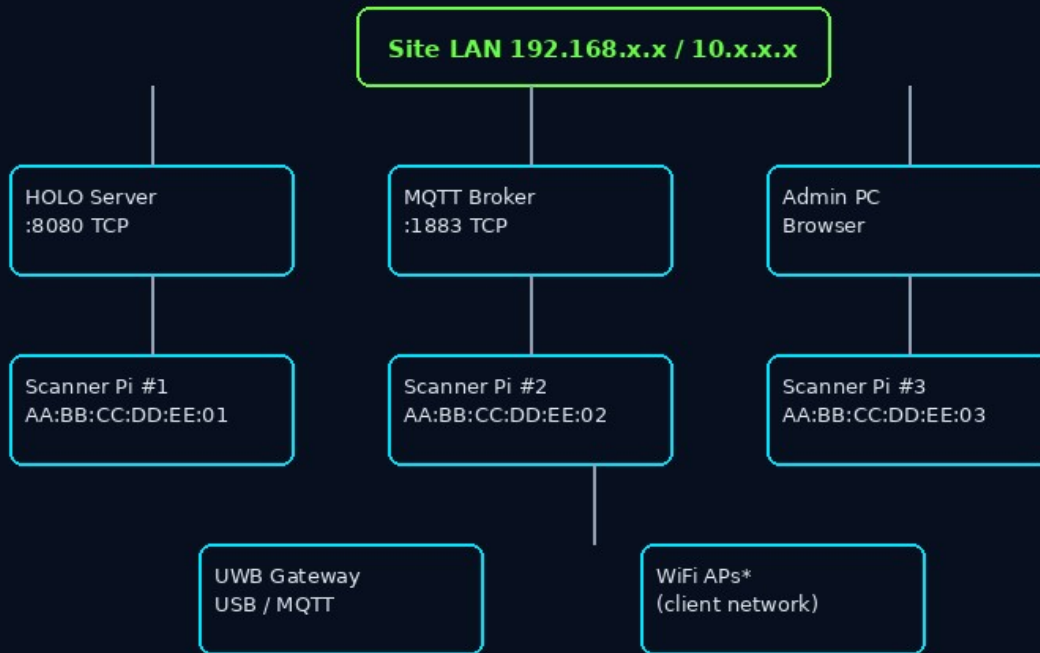
There is NO separate 'node server port' — scanners use HTTPS/HTTP to the main HOLO API. MQTT is optional for UWB/MQTT hardware profiles, not required for the scanner HTTP path.

Do nodes store server IP/port?

No. WifiNode and WifiAnchor database rows store MAC, name, and surveyed X/Y/Z only.

Each scanner Pi carries `BACKEND_URL` and `SCANNER_API_KEY` in environment or `config_local.py`.

Typical Site Network Layout



Firewall: allow Scanner Pis → Server:8080 and optional MQTT:1883. TLS recommended in production.

Default Demo Units (First Boot)

Admin: admin@holo-rtls.local / ChangeMe123! — CHANGE IN PRODUCTION

WifiNode anchors (Track A — /nodes):

AA:00:00:00:00:01 Demo Anchor N (0, 0, 2.5) m
AA:00:00:00:00:02 Demo Anchor E (20, 0, 2.5) m
AA:00:00:00:00:03 Demo Anchor S (20, 15, 2.5) m
AA:00:00:00:00:04 Demo Anchor W (0, 15, 2.5) m

Trackers (tags):

TAG_001 Operator Alpha PERSONNEL
TAG_002 Machine Cart B MACHINE
TAG_003 Sensor Pack C ENV_SENSOR

Hardware: Demo Mock Simulator (mock_data profile) — animates TAG_001-003

Scanner mock MACs (Track B testing with --mock):

WiFi: 11:22:33:44:55:01 ... 05
BLE: AA:00:00:00:00:01 ... 05 (demo overlap with anchor prefix)

Demo units are for training only. Replace with surveyed MACs/coordinates.

Adding WiFi Nodes & Anchors

Method 1 — Web UI (/nodes)

1. Login as Admin or Operator with MANAGE_NODE permission
2. Open Anchors & Nodes → Add Node
3. Enter: Name, MAC (AA:BB:CC:DD:EE:FF), X/Y/Z metres, type, coverage radius
4. Optional: Place on map via /?mode=setup → Place Nodes tool

Method 2 — REST API (Track A WifiNode)

POST /api/nodes Authorization: Bearer <JWT>

```
{ "name": "Gate A", "mac_address": "AA:BB:CC:DD:EE:FF",  
  "pos_x": 10, "pos_y": 5, "pos_z": 2.5, "node_type": 1 }
```

Method 3 — Scanner anchors (Track B WifiAnchor)

POST /api/scanner/anchors (requires JWT — Pi auto-register usually fails without it)

```
{ "mac_address": "AA:BB:CC:DD:EE:01", "name": "Scanner-North",  
  "real_x": 0, "real_y": 0, "real_z": 2.5, "tx_power": -40.0 }
```

Minimum 3 non-collinear scanner anchors with real_x/real_y for positioning.

Each Pi ANCHOR_MAC must match exactly one WifiAnchor row.

How Bluetooth + WiFi Positioning Works

BLUETOOTH (BLE)

Tag broadcasts iBeacon / Eddystone advert
Scanner Pi hci0 (bleak) listens passively
Receives: MAC, RSSI (dBm), optional name
No pairing — one-way receive only
signal_type = 2 in detection JSON

WiFi PROBE SCANNING

Phone/tag sends probe requests (802.11)
USB WiFi in monitor mode captures frames
Receives: MAC, RSSI, SSID, channel
No association to AP required
signal_type = 1 in detection JSON

POSITION CALCULATION (Track B)

Each anchor reports RSSI for the same tag MAC → server stores DetectionEvent rows
Path-loss model: $\text{distance} \approx 10^{((\text{tx_power} - \text{rssi}) / (10 \times n))}$ where $n \approx 2.0-3.5$
Trilateration: intersect 3+ distance circles → (x, y) fix, Kalman filter smooths motion
Accuracy: typically 3-15 m indoors (RSSI is noisy; geometry and calibration matter)
NOT sent over MQTT by default — scanner uses HTTP POST to /api/scanner/detections

Data Flow — WiFi / BLE / MQTT

1. EDGE SCAN

Scanner Pi hears WiFi probe / BLE advert
MAC + RSSI + type (1=WiFi, 2=BLE)

2. HTTP POST

POST /api/scanner/detections
Header: X-Scanner-Key
Body: anchor_mac + detections[]

3. TRILATERATION

≥ 3 WifiAnchor with real_x/y
RSSI $\rightarrow$ distance $\rightarrow$ (x,y) Kalman smooth

4. CORE SYNC

Upsert Tracker + history + SSE alert
Live Map + /api/positioning/live

MQTT IN (optional)

Topics: ble/rssi, wifi/rssi, rtls/positions
Hardware profiles map JSON fields

MQTT OUT

Topic: rtls/state_changes (QoS 1)
Alarm: rtls/alarms/trigger

SSE TO BROWSER

GET /api/stream/positions?token=JWT
events: snapshot, position_update, alert

NOT MQTT TO UI

Browsers use HTTPS + SSE, not MQTT

Scanner Pi Deployment (Track B)

Hardware per scanner node

Raspberry Pi 4 (2 GB+), 32 GB SD card, PoE HAT or industrial PSU
Alfa AWUS036NHA USB WiFi (monitor mode) — built-in Pi WiFi often insufficient
Optional USB BLE dongle if onboard hci0 is unreliable

Software install

```
sudo apt install bluetooth libbluetooth-dev python3-pip
pip install bleak scapy requests
sudo airmon-ng start wlan0 # creates wlan0mon
```

Environment (systemd or docker)

```
BACKEND_URL=http://192.168.1.10:8080
SCANNER_API_KEY=<match server .env>
ANCHOR_MAC=AA:BB:CC:DD:EE:01
WIFI_INTERFACE=wlan0mon
```

Run

```
sudo python3 scanner/main.py --wifi-iface wlan0mon
Dev/test without hardware: python3 scanner/main.py --mock
```

systemd auto-start

```
sudo cp scanner/holo-scanner.service /etc/systemd/system/
sudo systemctl enable holo-scanner && sudo systemctl start holo-scanner
```

Verify

Check Pi logs for 'Sent N detections'. On server, response includes `positions_computed > 0` when ≥ 3 anchors see the same tag MAC.

WiFi & Bluetooth Tags

WiFi tags / devices (Track B)

Any device emitting WiFi probe requests can be tracked by MAC address.
No pairing required. Register MAC as hardware_id in /trackers OR let scanner sync auto-create Tracker rows after first detection.

BLE tags / beacons

Supported via scanner bleak stack (Pi hci0) or hardware profiles:

- generic_ibeacon — UUID/Major/Minor beacons
- ruuvi_tag — environmental BLE sensor
- esp32_ble_gateway — ESP32 publishes RSSI to MQTT topic ble/rssi

UWB / serial tags (Track A)

Configure at /hardware → select profile (Qorvo, Sewio MQTT, etc.)
Register matching hardware_id at /trackers (e.g. TAG_004 or device serial).

Tag registration example

POST /api/trackers

```
{ "hardware_id": "11:22:33:44:55:AA", "assigned_name": "Worker 12",  
  "tag_type": 1, "category": 1 }
```

WiFi Access Point Guidance

HOLO does NOT configure APs automatically. APs provide RF environment only.

Recommendations:

- Use 2.4 GHz coverage for BLE + legacy WiFi probes (scanner dependent)
- Separate SSID/VLAN for scanner Pis vs guest/corporate WiFi
- Fixed IP or DHCP reservation for server and each scanner Pi
- Avoid AP placement collinear with scanner Pis (geometry matters)
- Track B: scanner Pi uses monitor-mode USB WiFi — NOT the AP management UI

Typical AP brands (UniFi, Aruba, Cisco, MikroTik CAP) all work as client WiFi.

Positioning accuracy depends on scanner Pi placement, not AP brand.

Installer Workflow — WiFi/BLE Site

- 1 A. Install HOLO server (.env, port 8080, change admin password)
- 2 B. Survey site → record anchor XY coordinates (metres)
- 3 C. Register WifiAnchor rows (JWT) with MAC + real_x/y/z
- 4 D. Deploy ≥ 3 Scanner Pis (BACKEND_URL, SCANNER_API_KEY, ANCHOR_MAC)
- 5 E. Calibrate TX power at 1 m reference distance per anchor
- 6 F. Register tags at /trackers (hardware_id = device MAC or tag ID)
- 7 G. Verify POST /api/scanner/detections → positions_computed > 0
- 8 H. Open Live Map — confirm live dots + SSE stream

Recommended Hardware BOM

HOLO Server:

- Linux VM or bare metal, 4 GB RAM+, Python 3.11-3.12
- Port 8080 open to site LAN; PostgreSQL optional for production

Track B Scanner node (each anchor location):

- Raspberry Pi 4 (2 GB+) + 32 GB SD
- Alfa AWUS036NHA or compatible monitor-mode USB WiFi
- USB BLE adapter if no reliable hci0
- PoE or industrial power; mount at surveyed coordinate

Track A UWB (optional high accuracy):

- Qorvo DWM1001 / Sewio / Pozyx per vendor plan
- USB-UART or MQTT broker

Tags:

- BLE iBeacon or Ruuvi for personnel/assets
- UWB tags for Track A
- WiFi: phones/laptops trackable by MAC (privacy policy required)

Network:

- Managed switch, optional MikroTik router (see MikroTik section)
- Optional Mosquitto MQTT broker for hardware integration
- Operator tablets/PCs on same LAN for browser access to :8080

Protocols — HTTP, SSE, MQTT

HTTP REST (primary data path)

Scanner → POST /api/scanner/detections (X-Scanner-Key auth)

Browser → all CRUD APIs (JWT Bearer)

External → POST /api/integrations/positions

SSE (browser live updates — NOT MQTT)

GET /api/stream/positions?token=<JWT>

Events: snapshot, position_update, alert, heartbeat

MQTT (optional hardware integration)

IN: Sewio rtls/positions, ESP32 ble/rssi, wifi/rssi, custom topics

OUT: rtls/state_changes (position JSON), rtls/alarms/trigger

Broker default localhost:1883 — configure MQTT_BROKER_HOST in .env

Detection JSON (scanner → server)

anchor_mac, detections[{mac_address, rssi, signal_type, ssid|adv_name}]

signal_type: 1=WiFi, 2=BLE

What is NOT used

Browsers do not subscribe to MQTT. Scanner Pis do not need MQTT unless you build a custom bridge. Track B positioning is HTTP-only by default.

MikroTik Routers — Role in HOLO-RTLS

VERDICT: Useful as site networking — NOT a drop-in HOLO anchor.

What MikroTik CAN do:

- Provide reliable WiFi coverage for staff/tablets accessing HOLO web UI
- VLAN-separate scanner Pis, server, and guest WiFi
- DHCP reservations for Scanner Pi MAC → fixed IP
- Firewall rules: Pis → server:8080, optional MQTT:1883
- CAPsMAN to manage multiple APs on large tunnels/plants

What MikroTik CANNOT replace (today):

- HOLO scanner daemon (needs monitor-mode USB WiFi or BLE on Pi)
- Built-in RouterOS WiFi does NOT feed RSSI trilateration to HOLO API
- No native integration in HOLO codebase (custom MQTT bridge possible)

Possible future path:

- ESP32 / custom script on RouterOS publishing RSSI JSON to MQTT
- Use Hardware profile 'esp32_wifi_scanner' or 'custom_mqtt'

Recommended: MikroTik for network + Raspberry Pi scanners for positioning

Alternative high-accuracy: UWB anchors (Track A) via /hardware profiles

MikroTik — Detailed Integrator Notes

MikroTik RouterOS is excellent for site networking (routing, VLANs, CAPsMAN, firewall, DHCP reservations) but is NOT a native HOLO positioning anchor.

Use MikroTik to:

- Backhaul scanner Pis and HOLO server on a reliable LAN
- Isolate OT/RTLS VLAN from corporate WiFi
- Reserve IPs for BACKEND_URL stability
- CAPsMAN: central management of many cAP/hAP units in tunnels/plants

Example DHCP reservation (RouterOS):

```
/ip dhcp-server lease add address=192.168.88.21 mac-address=DC:A6:32:XX:XX:01
```

Example firewall (allow scanners to HOLO):

```
/ip firewall filter add chain=forward src-address=192.168.88.0/24  
dst-address=192.168.88.10 dst-port=8080 protocol=tcp action=accept
```

Do not expect RouterOS built-in WiFi alone to feed HOLO trilateration.

Positioning still requires scanner daemon Pis or UWB/MQTT hardware profiles.

Advanced: RouterOS script or container publishing neighbor RSSI to MQTT could integrate via custom_mqtt hardware profile — requires custom development.

Troubleshooting Checklist

No positions on Live Map:

1. Confirm ≥ 3 WifiAnchor rows have real_x and real_y set
2. Confirm each Pi ANCHOR_MAC matches a registered anchor
3. curl -X POST http://server:8080/api/scanner/detections with test payload
4. Check SCANNER_API_KEY matches on Pi and server .env

Scanner cannot reach server:

- Ping server IP from Pi; verify firewall allows TCP 8080
- BACKEND_URL must include http:// and port 8080 (not 5000)

BLE not detecting:

- sudo hciconfig hci0 up; check bleak permissions (run as root if needed)

WiFi monitor mode fails:

- Use supported USB adapter; stop NetworkManager on wlan0

Demo mode works but production does not:

- Replace demo MACs; ensure tags are in range and broadcasting